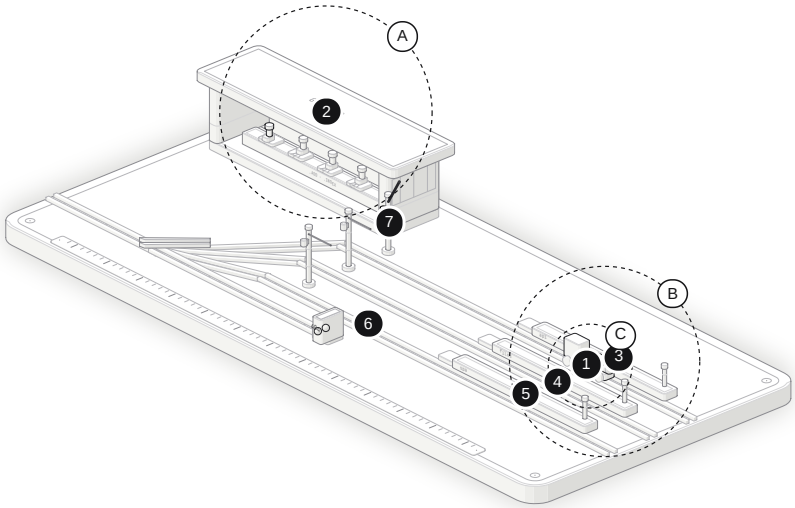
IHandler



What does every part of a GenHTTP server have in common?

Every part of GenHTTP is the same two-method IHandler: PrepareAsync warms the tree once before the first connection, HandleAsync answers the request. The lever frame is the builder, the points are the segment match, and every road ends somewhere — a response, or the dead spur that becomes a 404.

2 methods
PREPAREASYNC WARMS, HANDLEASYNC ANSWERS

4 roads
THREE HANDLERS AND ONE DEAD SPUR

null → 404
WHAT AN UNCLAIMED SEGMENT RETURNS

1 interface
NO SEPARATE MIDDLEWARE, EVER

0 special cases
NOT EVEN FOR THE ROOT OF THE TREE

PARTS

01 **The train**
One request, running the road the points set.

02 **The signal box**
The builder: .Add(segment, handler) hangs a platform on each branch.

03 **Platform api**
A routed segment — webservices, proxies, anything with a handler.

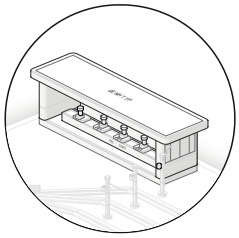
04 **Platform files**
Assets and trees: static files are handlers like any other.

05 **Platform spa**
A single-page app, with its fallback to index.html.

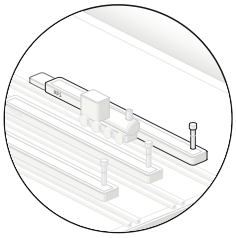
06 **The dead spur**
No handler claims it: HandleAsync returns null.

07 **Signals**
Arm dropped, lamp lit — the road that answered.

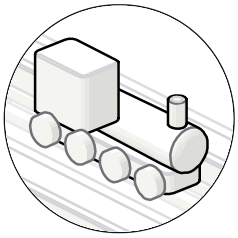
CLOSE-UPS



Ⓐ **The lever frame**
The builder API: segments, ordered fallbacks and the root index, composed into one handler by Concerns.Chain.

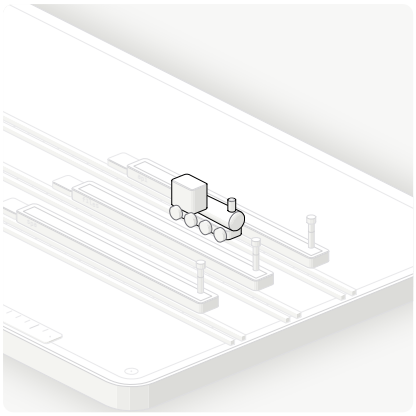


Ⓑ **A platform**
One .Add(segment, handler): multi-segment strings like "api/v1" split and merge into child layouts on the way in.



Ⓒ **The train**
The same IRequest reaches every platform — the Kestrel engine and the ASP.NET Core adapter hand their contexts over as exactly this.

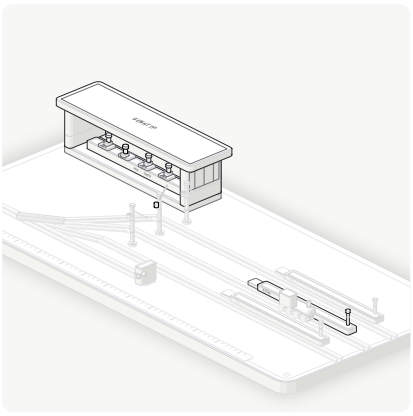
How it works



01

Two methods, one tree

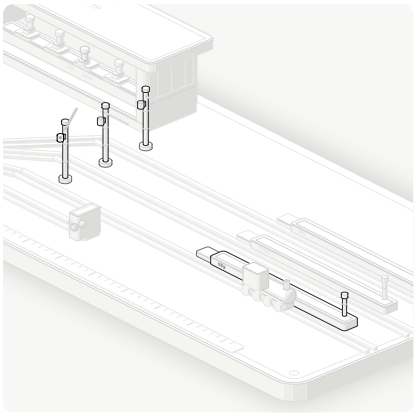
The train enters at the root. PrepareAsync warmed every road once before the first connection; now HandleAsync answers.



02

The points set

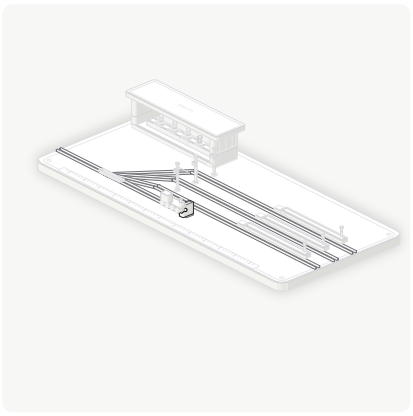
The segment match throws the points: .Add("files", handler) hung this platform on the branch when the tree was built.



03

Every road ends somewhere

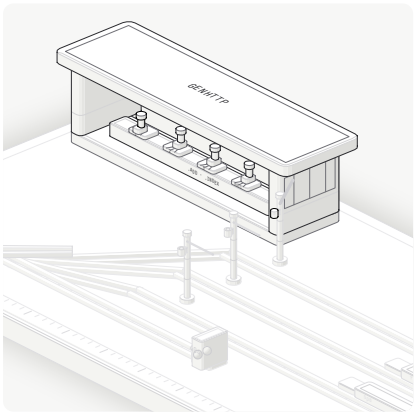
api, files, spa — webservices, asset trees, single-page apps. All handlers, so all of them compose.



04

The dead spur

No handler claims the road, so HandleAsync returns null — and the concern chain around the tree turns that into a 404.



05

One address per page

At the root the layout forces a trailing slash by redirecting, so the same content is never served at two URLs.

WHY ONE INTERFACE?

One two-method contract from the socket to your lambda means every module composes with every other by construction — no middleware interface distinct from handlers, no special case for the root, no hidden global state. That is also what makes the three-engine promise testable.

SPECIFICATION

Root	CoreRouter wraps your tree	Root page	.Index(handler), trailing slash forced
Order	compliance, errors, concerns, logging	Hosts	VirtualHosts.Create(), routed on Host
Segment	.Add(segment, handler)	Adapter	AspNetCore apps map onto handlers
Fallback	.Add(handler), ordered		